Speech Analysis and Resynthesis

Objective

This lab builds a speech processing pipeline, from time-domain statistics to spectral features, and inverts each representation back to a waveform. It ends with a source–filter separation by cepstral analysis.

Part 1: Initialization and Time-Domain Verification

Reproduce the baseline environment and basic signal statistics.

1. **Environment Setup:** Work in a Jupyter notebook. Import `librosa` (McFee et al. 2015) and its `display` module, along with `numpy`, `pandas`, `pathlib`, `scipy.signal`, `sklearn.preprocessing`, `matplotlib.pyplot`, and `IPython.display`.
2. **Parameters:** Set the sampling frequency to 16 kHz and the Fast Fourier Transform (FFT) size to 512. Use a Hann window of 0.03 s with a hop of 0.01 s. The features have 80 mel bands and 13 Mel-frequency cepstral coefficients (MFCCs). Optionally, zero-pad the FFT by a factor of 4 for smoother plots. Zero-padding interpolates the spectrum without improving its resolution. Whichever FFT size you choose, use it in all subsequent calls.
3. **Frame Lengths:** Convert the durations into samples. You should obtain `WINDOW: 480, HOP: 160`.
4. **Statistics:** Write `print_stats(xs, fs)`, which prints the extrema, mean, standard deviation, duration, and shape of an audio array.
5. **Data:** Load `Ses01F_impro05_F002.wav` (or a file of your choice) at 16 kHz with `librosa.load`. Plot its waveform with `librosa.display.waveshow`. Play it with `IPython.display.Audio`. Then normalize its peak to 1 and keep a voiced excerpt of about 1 s (samples 5000 to 21000). The rest of the lab uses this excerpt.

Part 2: Spectral Analysis and Feature Extraction

Compute spectral features according to the hyperparameters given as constants.

1. **DFT:** Compute the Discrete Fourier Transform (DFT) of the excerpt with `numpy.fft`. Plot its magnitude over the positive frequencies, on linear and dB scales, and check that the inverse DFT returns the signal.
2. **STFT:** Compute the complex Short-Time Fourier Transform (STFT) with `librosa.stft`. Display its magnitude in dB as a spectrogram, with linear and logarithmic frequency axes.
3. **Filter Design** (*optional*): Design a 1 kHz low-pass filter with the windowed-sinc method (Smith 1997, chap. 16).
   - Applied to the ideal brick-wall response, `numpy.fft.irfft` returns a sinc wrapped around sample 0. Center it with `numpy.roll`, truncate it to $M+1$ samples, apply a Hann window, and set its DC gain to 1.
   - Choose M from the transition bandwidth BW, as a fraction of the sampling frequency, with $M \approx 4/BW$. Compare the responses (in dB) of the ideal, truncated, and windowed kernels.
   - Filter the excerpt by FFT convolution. Zero-pad both signals to $N+M$ samples to avoid a circular convolution, then compensate for the delay of $M/2$ samples. Listen to the result, and compare the spectrum of a vowel frame before and after filtering.
4. **Mel Spectrogram:** Build an 80-band mel filterbank with `librosa` and plot about 20 of its bands. Apply it to the power spectrogram $|X|^2$ and display the result in dB. Pass `sr` explicitly, since `librosa` assumes 22050 Hz by default.
5. **MFCCs:** Extract 13 MFCCs from the log-power mel spectrogram. Standardize each coefficient with `StandardScaler`, which performs cepstral mean and variance normalization. Transpose the matrix first, because `scikit-learn` expects (samples, features). Keep the raw MFCCs for Part 3.

Part 3: Signal Resynthesis

Invert each representation back to the time domain.

1. **Inverse STFT:** Reconstruct the signal from the complex STFT with `librosa.istft`, passing `length` so that both arrays match. Why is the reconstruction near-perfect?
2. **Griffin-Lim:** Estimate the phase from the STFT magnitude alone with `librosa.griffinlim` (Griffin and Lim 1984).
    - Optionally, implement the algorithm yourself. Start from a random phase, then alternate an inverse STFT and an STFT, restoring the original magnitude at each iteration. Compare your result with `librosa.griffinlim`, and plot the spectral convergence (item 4) over the iterations.
3. **Feature Inversion:** Recover an STFT magnitude from the mel spectrogram, then from the MFCCs, with `librosa.feature.inverse`. Resynthesize both with Griffin-Lim, and compare 13 with 80 MFCCs.
4. **Evaluation:**
    - For the inverse STFT, report the maximum absolute error and the signal-to-noise ratio (SNR) in dB.
    - Magnitude-only reconstructions (Griffin-Lim, mel spectrogram, and MFCCs) recover the phase up to ambiguities, such as a polarity flip. Their waveforms thus differ even when they sound alike. Instead, use the spectral convergence, where $\hat{X}$ is the STFT of the resynthesis.

$$\mathrm{SC} = \frac{\| \, |X| - |\hat{X}| \, \|_F}{\| \, |X| \, \|_F}$$

    - Overlay the original and resynthesized waveforms over about 30 ms, and listen to every reconstruction.

Part 4: Source–Filter Separation via Homomorphic Analysis

Separate the glottal source from the vocal tract filter with the real cepstrum of a voiced frame (Rabiner and Schafer 2007, chap. 5).

1. **Cepstrum and Liftering:** Compute the real cepstrum of a vowel frame, $c[n] = \mathrm{DFT}^{-1}\{\log|X[k]|\}$, and plot it against quefrency. A low-time lifter keeps the coefficients below a cutoff shorter than the pitch period, such as 2.5 ms (40 samples). Since the cepstrum is even, apply it symmetrically at both ends of the array. The complementary high-time lifter keeps the rest.

2. **Envelope and Formants:** The DFT of the low-time part gives the log-spectral envelope. Estimate F_1 to F_3 from its local maxima with `scipy.signal.find_peaks`. Beware of a spurious peak just above the first harmonic, caused by the ringing of the lifter. Compare this envelope with the one implied by the 13 MFCCs of the frame.

3. **Excitation and f_0:** The DFT of the high-time part gives the log-excitation spectrum. Estimate $f_0 = f_s/n_0$ from the highest cepstral peak n_0 between 2.5 and 20 ms (50 to 400 Hz). Check it against `librosa.pyin`.

4. **Resynthesis:** Check that the log-envelope and the log-excitation add up exactly to the log spectrum, and explain why. Lifter every STFT frame, then resynthesize the filter and the source with the original phase. Optionally, drive the envelope with a pulse train at the pYIN f_0, then with white noise, as a basic cepstral vocoder.

5. **Discussion:**

   - Why does Griffin-Lim sound "phasey," even from the exact magnitude?
   - What do the mel filterbank and the truncation to 13 MFCCs lose? Why are 13 coefficients nonetheless enough for speech recognition?
   - How do the lifter cutoff and the frame choice affect the formant and f_0 estimates?
   - Which component would you modify to change the pitch? Which one determines the vowel?

References

Griffin, Daniel and Jae Lim (1984). "Signal estimation from modified short-time Fourier transform." In: *IEEE Transactions on Acoustics, Speech, and Signal Processing* 32.2, pp. 236–243. DOI: 10.1109/TASSP.1984.1164317.

McFee, Brian et al. (2015). "librosa: Audio and music signal analysis in Python." In: *Proceedings of the 14th Python in Science Conference (SciPy 2015)*. DOI: 10.25080/Majora-7b98e3ed-003.

Rabiner, Lawrence R. and Ronald W. Schafer (2007). "Introduction to digital speech processing." In: *Foundations and Trends in Signal Processing* 1.1–2, pp. 1–194. DOI: 10.1561/2000000001.

Smith, Steven W. (1997). *The scientist and engineer's guide to digital signal processing*. California Technical Publishing. URL: https://www.dspguide.com.